DATA STRUCTURES & ALGORITHMS

MODULE I — INTRODUCTION

Advanced study notes with clear explanations, Python examples, complete outputs, dry runs, recurrence-solving methods, and exam-focused summaries.

Topics covered

1. Introduction to Data Structures
2. Algorithm design paradigms and motivation
3. Algorithmic efficiency and runtime analysis
4. Asymptotic notation: O, Ω, Θ, o, ω
5. Recurrences: substitution, recursion tree, and Master Theorem

Prepared for learning and revision • Python 3

How to use these notes

Read each concept, study the pseudocode or Python program, then manually trace the example. Runtime analysis describes how resource use grows with input size; it is not a promise of exact seconds on every computer.

> **Important distinction:** A data structure organizes data. An algorithm specifies steps to solve a problem. They work together, but they are not the same thing.

Contents

• 1. Data structures: meaning, operations, classifications, and examples

• 2. Algorithm design paradigms: motivation and common strategies

• 3. Algorithmic efficiency: time, space, input size, and cases

• 4. Runtime analysis: counting operations and analyzing loops

• 5. Asymptotic notation: formal definitions and examples

• 6. Recurrences: meaning and common forms

• 7. Substitution method: guess, induction hypothesis, prove

• 8. Recursion-tree method: levels, work per level, total

• 9. Master Theorem: cases, conditions, and worked examples

• 10. Python examples with complete outputs and dry runs

• 11. Common mistakes, formula sheet, and practice questions

1. Introduction to Data Structures

1.1 What is a data structure?

A **data structure** is a systematic way of storing and organizing data so that operations such as access, search, insertion, deletion, and update can be performed effectively. The right structure depends on the operations a program performs most often.

Example: storing marks

```
marks = [85, 72, 91, 68, 77]
print(marks[0])
print(marks[2])
```

Output

```
85
91
```

Python lists are zero-indexed: index 0 refers to the first item, index 1 to the second, and so on. Accessing an item by a known list index is typically O(1), while searching for an arbitrary value in an unsorted list is O(n).

1.2 Common categories

Category	Meaning	Examples	Typical use
Primitive/basic types	Single values or language-level building blocks	int, float, bool, char-like strings	Represent a number, flag, or text value
Linear structures	Elements arranged in a sequence	array/list, linked list, stack, queue	Ordered collections and processing sequences
Non-linear structures	Relationships branch or form networks	tree, heap, graph	Hierarchies, priority processing, networks
Hash-based structures	Use a hash function to locate keys	Python dict, set	Fast average membership and key lookup
Static vs dynamic	Capacity fixed vs able to grow/shrink	fixed-size array vs Python list	Memory/layout and flexibility choices

1.3 Core operations and why structure choice matters

Operation	Meaning	Examples of typical costs
Access	Read an item at a known position	Array/list index: O(1); linked list by position: O(n)
Search	Find whether a value/key exists	Unsorted list: O(n); hash set: average O(1)
Insertion	Add an item	List append: amortized O(1); insert at front of list: O(n)
Deletion	Remove an item	List removal by value may be O(n); hash removal average O(1)
Traversal	Visit every item	Usually O(n) for n items

These costs are typical, not universal guarantees for every implementation. For example, Python dictionary operations are average O(1), but their worst-case behavior can be different.

1.4 Data structure example: list vs set

```
numbers = [4, 7, 4, 9]
unique_numbers = set(numbers)

print(numbers)
print(unique_numbers)
print(7 in numbers)
print(7 in unique_numbers)
```

Possible output (set display order can vary)

```
[4, 7, 4, 9]
{9, 4, 7}
True
True
```

A list preserves sequence and duplicates. A set stores unique hashable values and is useful for membership checks. A set does not promise a meaningful display order.

1.5 Abstract Data Types (ADTs)

An **Abstract Data Type** defines what operations a collection supports, without requiring one particular implementation. A stack ADT, for example, supports push, pop, and peek with last-in, first-out behavior. It can be implemented using a Python list or a linked list.

ADT	Rule	Common operations	Example
Stack	LIFO: last in, first out	push, pop, peek	Undo history, function calls
Queue	FIFO: first in, first out	enqueue, dequeue, front	Task scheduling, waiting lines
Priority queue	Highest/lowest priority removed first	insert, extract-min/max	Scheduling, graph algorithms
Map/dictionary	Key maps to a value	put/update, get, delete	Student ID → student record

2. Algorithm Design Paradigms — Motivation

An **algorithm** is a finite, precise sequence of steps that transforms input into output to solve a problem. A design paradigm is a general strategy for constructing algorithms. Paradigms help us avoid solving every new problem from scratch.

2.1 Properties of a good algorithm

- **Input:** accepts zero or more well-defined inputs.
- **Output:** produces at least one result relevant to the problem.
- **Definiteness:** each step is clear and unambiguous.
- **Finiteness:** it terminates after a finite number of steps.
- **Effectiveness:** each step can actually be carried out.
- **Correctness:** it produces the required answer for every valid input.
- **Efficiency:** it uses reasonable time and memory for the problem size.

2.2 Why use design paradigms?

A paradigm offers a reusable way to break down a problem, reason about correctness, and estimate performance. Different paradigms can solve the same problem with different trade-offs. Choosing a paradigm is not just about speed: correctness, memory, simplicity, and constraints also matter.

2.3 Major paradigms

Paradigm	Core idea	Example	When it is useful
Brute force	Try all candidates or directly follow the definition	Linear search; checking all pairs	Small input, simple baseline, no better structure known
Divide and conquer	Divide into smaller independent subproblems, solve them, combine	Merge sort; binary search	Subproblems shrink predictably and can be combined
Decrease and conquer	Solve a smaller version, then extend the result	Insertion sort; some recursive algorithms	A solution can be built from a smaller input
Greedy	Choose the best-looking local option at each step	Kruskal's minimum spanning tree	When a proof shows local choices lead to a global optimum
Dynamic programming	Store answers to overlapping subproblems	Fibonacci with memoization; knapsack	Subproblems repeat and have optimal substructure
Backtracking	Build a candidate and undo choices that cannot work	N-Queens; Sudoku	Constraint satisfaction and search
Branch and bound	Search candidates while pruning branches using bounds	Exact travelling-salesperson search	Optimization problems where bounds can rule out inferior solutions
Transform and conquer	Change representation or reduce to a known problem	Heap sort; balancing/searching structures	A transformation simplifies the main task

2.4 Example: brute-force search vs binary search

To find a target in an unsorted list, linear search checks items one by one. It can take O(n) time. Binary search repeatedly halves the search interval, taking O(log n), but it requires sorted data (or another valid monotonic ordering).

```
def linear_search(items, target):
    for i, value in enumerate(items):
        if value == target:
            return i
    return -1

def binary_search(sorted_items, target):
    left = 0
    right = len(sorted_items) - 1

    while left <= right:
        mid = (left + right) // 2
        if sorted_items[mid] == target:
            return mid
        elif sorted_items[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

data = [3, 8, 12, 17, 25, 31, 44]
print(linear_search(data, 25))
print(binary_search(data, 25))
```

Output

```
4
4
```

Both return index 4 for this input. The advantage of binary search becomes clearer for large sorted collections: it discards about half of the remaining candidates at each step.

2.5 Divide and conquer pattern

1. Divide the original problem into smaller subproblems.

2. Solve each subproblem, often recursively.

3. Combine the subproblem solutions into the overall result.

Merge sort follows this pattern: split a list, sort each half, then merge the sorted halves. If two halves each take T(n/2) time and merging takes linear time, its recurrence is T(n) = 2T(n/2) + Θ(n), which solves to Θ(n log n).

3. Algorithmic Efficiency

3.1 What is efficiency?

Algorithmic efficiency describes how the resources required by an algorithm grow as input size increases. The two principal resources are **time** (number of primitive operations or execution work) and **space** (memory use).

• **Time complexity:** how operation count grows with input size n.

• **Space complexity:** how memory use grows with n.

• **Auxiliary space:** extra memory used by the algorithm, often excluding the input itself.

• **Input size:** the parameter that measures the problem, such as number of list items, number of graph vertices, or number of bits in an integer.

> Complexity is generally not the same as elapsed seconds. Hardware, language, compiler/interpreter, input distribution, and implementation constants affect real runtime. Asymptotic analysis focuses on growth as input becomes large.

3.2 Best, average, and worst cases

Case	Meaning	Linear search example
Best case	Most favorable valid input of size n	Target is first: Θ(1)
Worst case	Most expensive valid input of size n	Target is last or absent: Θ(n)
Average case	Expected cost under a stated input probability model	For uniformly positioned successful target: about (n+1)/2 checks, Θ(n)

Average-case analysis needs an assumption about how inputs are distributed. Without a probability model, an average-case claim may be underspecified.

3.3 Count operations, not seconds

A common approach is to select a basic operation (such as a comparison or assignment) and count how many times it executes. Then simplify the count using asymptotic notation.

```
def print_items(items):
    for item in items:
        print(item)

print_items([10, 20, 30])
```

Output

```
10
20
30
```

For n items, the loop body runs n times. If each iteration performs constant work, T(n) = an + b for constants a and b, so T(n) = Θ(n).

3.4 Common growth rates

Complexity	Name	Growth intuition	Example
O(1)	Constant	Does not grow with n	Read list item by index
O(log n)	Logarithmic	Input is repeatedly divided by a constant factor	Binary search
O(n)	Linear	Work proportional to number of items	One list traversal
O(n log n)	Linearithmic	About log n levels, each doing linear work	Merge sort
O(n²)	Quadratic	All pairs or nested n-sized loops	Simple comparison of every pair
O(n³)	Cubic	Three nested dimensions of work	Basic triple-loop computation
O(2■)	Exponential	Work roughly doubles with each added input unit	Enumerating all subsets
O(n!)	Factorial	Enumerating permutations	Brute-force route ordering

3.5 Python example: linear vs quadratic loops

```
def linear_work(n):
    count = 0
    for i in range(n):
        count += 1
    return count

def quadratic_work(n):
    count = 0
    for i in range(n):
        for j in range(n):
            count += 1
    return count

print(linear_work(5))
print(quadratic_work(5))
```

Output

```
5
25
```

The first function increments the counter n times. The second increments it n × n = n² times. For n = 5, these are 5 and 25 operations of the counted kind. Actual machine instructions include overhead, but the growth patterns remain linear and quadratic.

3.6 Analyzing common loop patterns

```
# A: one loop
for i in range(n):
    do_constant_work()

# B: nested loops
for i in range(n):
    for j in range(n):
        do_constant_work()

# C: repeatedly halve a variable
i = n
while i > 1:
    i //= 2
```

Pattern	Iteration count	Complexity
A	n	Θ(n)
B	n × n	Θ(n²)
C	Approximately log■(n)	Θ(log n)

3.7 Consecutive vs nested loops

Consecutive loops add their costs; nested loops multiply iteration counts when each loop's full range runs for every outer iteration.

```
for i in range(n):
```

```
    work()             # n times

for j in range(n):
    for k in range(n):
        work()         # n^2 times
```

Total work is proportional to $n + n^2$. The n^2 term dominates for large n, so the overall complexity is $\Theta(n^2)$, not $\Theta(n + n^2)$ as the final simplified asymptotic classification.

3.8 Space complexity and recursion

An algorithm can be fast but use a lot of memory, or use little memory but take longer. A loop-based sum usually uses $\Theta(1)$ auxiliary space. A recursive algorithm may use $\Theta(\text{depth})$ call-stack space even when each call creates no large data structure.

```
def sum_to_n(n):
    total = 0
    for value in range(1, n + 1):
        total += value
    return total

print(sum_to_n(5))
```

Output

```
15
```

The loop-based implementation uses constant extra variables: auxiliary space $\Theta(1)$. Its time complexity is $\Theta(n)$.

4. Runtime Analysis: Step-by-Step

4.1 A systematic method

1. Identify the input-size parameter n.
2. Choose the operation(s) to count (comparisons, assignments, recursive calls, etc.).
3. Count how many times each operation executes.
4. Write a mathematical function T(n).
5. Keep dominant terms and ignore constant factors for asymptotic classification.
6. State assumptions, such as whether input is sorted and whether the analysis is worst-case.

4.2 Example: sum of first n positive integers

```
def sum_n(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

print(sum_n(5))
```

Output

```
15
```

The loop executes n times. If the body costs constant time, $T(n) = an + b$ for fixed constants a and b. Therefore, $T(n) = \Theta(n)$. There is also a mathematical formula $n(n+1)/2$, which can compute the result in $\Theta(1)$ arithmetic operations under the usual unit-cost model.

```
def sum_n_formula(n):
    return n * (n + 1) // 2

print(sum_n_formula(5))
```

Output

```
15
```

> The formula is constant-time only under a model where arithmetic on the numbers costs constant time. For arbitrarily large integers, the bit-length of the numbers affects arithmetic cost. Introductory algorithm courses usually use the unit-cost model unless specified otherwise.

4.3 Example: triangular nested loop

```
def count_pairs(n):
    count = 0
    for i in range(n):
        for j in range(i):
            count += 1
    return count

print(count_pairs(5))
```

Output

```
10
```

For i = 0, the inner loop runs 0 times; for i = 1, once; then 2, 3, and 4 times. Total = 0 + 1 + 2 + 3 + 4 = n(n–1)/2 = Θ(n²). The exact count for n = 5 is 10.

4.4 Amortized analysis (advanced note)

Amortized analysis studies the average cost per operation over a sequence of operations, without requiring a probability distribution. For example, appending to a dynamic array/list is usually amortized O(1): most appends are constant time, while occasional resizing copies many elements. The expensive resize is spread over many cheap appends.

Amortized analysis differs from average-case analysis: average-case analysis uses assumptions about the probability of inputs; amortized analysis provides a bound across a sequence of operations.

5. Asymptotic Notations

Asymptotic notation describes how a function grows as n becomes large. It abstracts away machine-dependent constants and lower-order terms.

5.1 Big O — asymptotic upper bound

f(n) = O(g(n)) if there exist constants c > 0 and n■ such that 0 ≤ f(n) ≤ c·g(n) for every n ≥ n■. Big O says that f does not grow faster than a constant multiple of g eventually. It is commonly used for upper bounds, including worst-case upper bounds.

5.2 Big Omega — asymptotic lower bound

f(n) = Ω(g(n)) if there exist constants c > 0 and n■ such that 0 ≤ c·g(n) ≤ f(n) for every n ≥ n■. It is a lower bound on growth.

5.3 Big Theta — tight asymptotic bound

f(n) = Θ(g(n)) when f(n) is both O(g(n)) and Ω(g(n)). Equivalently, for suitable positive constants c■, c■ and sufficiently large n, c■g(n) ≤ f(n) ≤ c■g(n). Θ gives a tight growth rate.

5.4 Little o and little omega

f(n) = o(g(n)) means f grows strictly slower than g in the limiting sense: f(n)/g(n) → 0. f(n) = ω(g(n)) means f grows strictly faster: f(n)/g(n) → ∞. These are stricter than O and Ω.

Notation	Meaning	Example
O(g(n))	Asymptotic upper bound	3n² + 2n + 1 = O(n²)
Ω(g(n))	Asymptotic lower bound	3n² + 2n + 1 = Ω(n²)
Θ(g(n))	Tight bound (both upper and lower)	3n² + 2n + 1 = Θ(n²)
o(g(n))	Strictly smaller growth	n = o(n²)
ω(g(n))	Strictly larger growth	n² = ω(n)

5.5 Worked example: remove lower-order terms

Let f(n) = 7n² + 4n + 9. For n ≥ 1, the n² term dominates the growth. An upper bound is f(n) ≤ 20n² for n ≥ 1 because 4n ≤ 4n² and 9 ≤ 9n², so f(n) ≤ (7+4+9)n² = 20n². Also f(n) ≥ 7n². Thus f(n) = Θ(n²).

> Common shorthand: keep the fastest-growing term and ignore constant multipliers. This is a method for finding a likely tight class; the formal definitions explain why it is valid.

5.6 Comparing growth rates

From slower to faster (typical)	Notes
$\Theta(1) < \Theta(\log n)$	Constant vs logarithmic
$\Theta(\log n) < \Theta(n)$	Logarithm grows slower than linear
$\Theta(n) < \Theta(n \log n)$	Extra logarithmic factor
$\Theta(n \log n) < \Theta(n^2)$	For sufficiently large n
$\Theta(n^2) < \Theta(n^3)$	Higher polynomial degree grows faster
$\Theta(n^k) < \Theta(c^n)$ for $c > 1$	For fixed k and exponential base $c > 1$
$\Theta(c^n) < \Theta(n!)$	For fixed $c > 1$, factorial eventually dominates

5.7 Common misconceptions

• Big O is not automatically the exact complexity; $O(n^2)$ is also a valid upper bound for a $\Theta(n)$ algorithm, but it is less informative.

• Best, average, and worst case describe input cases; O, Ω, and Θ describe bounds on functions. They are related but not interchangeable.

• Drop constants and lower-order terms for asymptotic growth, not when an exact operation count is requested.

• Binary search is not O(log n) on an unsorted list unless the required ordering is established by some additional process.

• Two algorithms with the same asymptotic class can perform differently for practical input sizes because constants and implementation details matter.

6. Recurrences and Recursive Algorithms

6.1 What is a recurrence?

A recurrence relation defines a function using its values on smaller inputs. Runtime recurrences are used when an algorithm calls itself recursively. A recurrence normally needs a base case to define the smallest input.

General divide-and-conquer form: $T(n) = aT(n/b) + f(n)$, where a is the number of recursive subproblems, n/b is the approximate size of each subproblem, and f(n) is the work done outside the recursive calls.

6.2 Example: recursive countdown

```
def countdown(n):
    if n == 0:
        print("Done")
        return
    print(n)
    countdown(n - 1)

countdown(3)
```

Output

```
3
2
1
Done
```

The function makes one recursive call with n–1 until n = 0. If each call does constant work, the recurrence is $T(n) = T(n–1) + \Theta(1)$, with $T(0) = \Theta(1)$. There are n+1 calls, so $T(n) = \Theta(n)$, and the call stack uses $\Theta(n)$ space.

6.3 Example: recurrence for merge sort

Merge sort divides an input into two halves, recursively sorts both, then merges them in linear time. Thus $T(n) = 2T(n/2) + cn$, with $T(1) = d$. For simplicity, assume n is a power of two. The solution is $\Theta(n \log n)$. The methods in the next sections explain how to derive this.

6.4 Why solve recurrences?

• To analyze recursive algorithms rather than guess their cost.

• To compare algorithms that split work differently.

• To prove upper or lower bounds on runtime.

• To understand how the number and size of recursive subproblems affect growth.

7. Substitution Method

The substitution method solves a recurrence by **guessing a bound** and then **proving it by induction**. The name can be confusing: it usually means substituting the induction hypothesis into the recurrence.

7.1 Standard process

1. Guess a plausible asymptotic bound from expansion or intuition.

2. State an induction hypothesis for smaller input sizes.

3. Substitute the hypothesis into the recurrence.

4. Choose constants so the resulting inequality proves the guess.

5. Verify base cases and conclude the bound.

7.2 Worked example: T(n) = T(n/2) + 1

Assume n is a power of two, T(1) = 1, and for n > 1, T(n) = T(n/2) + 1. The expected answer is Θ(log n).

Upper bound: O(log n)

Guess T(n) ≤ c log■ n + d for suitable constants. If the hypothesis holds for n/2, then:

```
T(n) = T(n/2) + 1
     <= c log2(n/2) + d + 1
      = c(log2(n) - 1) + d + 1
      = c log2(n) + d + (1 - c)
```

If c ≥ 1, then 1–c ≤ 0, so T(n) ≤ c log■ n + d. Choose d to make the base case true. This proves an O(log n) upper bound.

Lower bound: Ω(log n)

Each recursive step adds 1, and the recursion takes log■ n steps to reach 1. Therefore at least a constant times log n work is done: T(n) = Ω(log n). Together, T(n) = Θ(log n).

7.3 Worked example: T(n) = 2T(n/2) + n

Assume T(1) = 1 and n is a power of two. We expect Θ(n log n). Prove an upper bound using induction. Suppose T(n/2) ≤ c(n/2)log■(n/2) + dn/2, a form equivalent to a candidate c n log■ n plus a linear adjustment.

Substitute:

```
T(n) = 2T(n/2) + n
     <= 2[c(n/2) log2(n/2) + d(n/2)] + n
      = c n(log2 n - 1) + d n + n
      = c n log2 n + (d + 1 - c)n
```

Choose c ≥ d + 1 so that (d + 1 – c)n ≤ 0. With constants selected to satisfy the base case, this establishes T(n) = O(n log n). A corresponding lower-bound argument (or recursion-tree analysis) gives Ω(n log n), so T(n) = Θ(n log n).

> For substitution proofs, the induction hypothesis often needs strengthening. If a direct guess leaves an unwanted positive term, adjust the bound (for example, use cn–d rather than cn) and check the base case.

7.4 Recurrence T(n) = T(n–1) + n

Expand the recurrence: T(n) = T(n–1)+n = T(n–2)+(n–1)+n = ... = T(1)+2+3+...+n. The sum is n(n+1)/2 – 1, up to the chosen base constant. Therefore T(n) = Θ(n²).

7.5 When substitution is useful

• When you already suspect the answer and want a rigorous proof.

• When the recurrence does not fit the Master Theorem.

• When the theorem gives only a bound and a more tailored proof is needed.

• When induction details and constants matter.

8. Recursion-Tree Method

A recursion tree visualizes a recurrence as levels of recursive calls. Each node represents one subproblem; its cost is the non-recursive work at that node. Add the costs at each level, then sum across all levels.

8.1 General procedure

1. Draw the root problem and its immediate recursive children.

2. Write the non-recursive work at each node.

3. Find the total work at each depth/level.

4. Find the number of levels until the base case.

5. Sum the level costs, including leaf costs when relevant.

8.2 Worked example: T(n) = 2T(n/2) + n

At the root, the non-recursive work is n. It creates two subproblems of size n/2, each doing n/2 work, so the next level totals n. At depth 2 there are four problems of size n/4, so the level total is again n.

Level	Number of subproblems	Size per subproblem	Work per node	Total level work
0	1	n	n	n
1	2	n/2	n/2	n
2	4	n/4	n/4	n
i	2^i	n/2^i	n/2^i	n
log■ n	n	1	Θ(1)	Θ(n)

There are log■ n levels before subproblem size reaches 1. Each level costs Θ(n), so the total is Θ(n) × Θ(log n) = Θ(n log n). Leaf work totals Θ(n), which does not change the final bound.

8.3 Worked example: T(n) = T(n/2) + 1

There is one subproblem per level. Each level costs 1, and the recursion depth is log■ n. Total work is 1 + 1 + ... + 1 for log■ n levels, so T(n) = Θ(log n).

8.4 Worked example: T(n) = 3T(n/2) + n

At level i there are 3^i subproblems, each of size n/2^i. If each node does linear non-recursive work, each node costs Θ(n/2^i), so total level work is Θ((3/2)^i n). This increases by a factor of 3/2 each level. The leaves occur at depth log■ n; there are 3^(log■ n) = n^(log■ 3) leaves. The leaf level dominates, giving Θ(n^(log■ 3)) because log■ 3 ≈ 1.585 > 1.

8.5 Pitfalls

• Do not count only one branch; all recursive calls contribute.

• Distinguish cost per node from total cost across a level.

• Count the leaves and their costs if they are significant.

• Do not assume every level costs the same; inspect how the number of nodes and work per node change.

9. Master Theorem / Master Method

The Master Theorem provides asymptotic solutions for recurrences of the form T(n) = aT(n/b) + f(n), where a ≥ 1 is a constant number of subproblems, b > 1 is a constant shrink factor, and f(n) is the non-recursive work per call. Define p = log_b(a). Compare f(n) with n^p.

9.1 The three standard cases

Case	Condition (simplified standard form)	Solution	Interpretation
1	f(n) = O(n^(p–ε)) for some ε > 0	T(n) = Θ(n^p)	Leaves/subproblem work dominates
2	f(n) = Θ(n^p log^k n), k ≥ 0	T(n) = Θ(n^p log^(k+1) n)	Each level has comparable work; accumulate over log n levels
3	f(n) = Ω(n^(p+ε)) for some ε > 0, plus regularity condition	T(n) = Θ(f(n))	Root/top-level work dominates

Here ε is any positive constant. For Case 3, the regularity condition is commonly written a f(n/b) ≤ c f(n) for some constant c < 1 and sufficiently large n. The simplified cases above cover many textbook problems but are not every possible recurrence.

9.2 How to apply it

1. Put the recurrence into T(n) = aT(n/b) + f(n).

2. Identify a, b, and f(n).

3. Calculate p = log_b a.

4. Compare f(n) with n^p.

5. Choose the applicable case and state the result.

9.3 Example 1: T(n) = 2T(n/2) + 1

a = 2, b = 2, f(n) = Θ(1). Then n^(log■ 2) = n. Since f(n) = O(n^(1–ε)) for ε = 1, Case 1 applies. Therefore T(n) = Θ(n).

9.4 Example 2: T(n) = 2T(n/2) + n

a = 2, b = 2, f(n) = Θ(n), and n^(log■ 2) = n. Thus f(n) = Θ(n^1), matching Case 2 with k = 0. Therefore T(n) = Θ(n log n).

9.5 Example 3: T(n) = 2T(n/2) + n²

a = 2, b = 2, so n^(log■ 2) = n. Here f(n) = n², which is polynomially larger than n. The regularity condition holds because 2(n/2)² = n²/2 ≤ c n² for c = 1/2. Case 3 applies, so T(n) = Θ(n²).

9.6 Example 4: T(n) = 4T(n/2) + n²

a = 4, b = 2, so p = log■ 4 = 2. The non-recursive term f(n) = Θ(n²) matches n^p. Case 2 gives T(n) = Θ(n² log n).

9.7 Example 5: T(n) = 3T(n/2) + n

a = 3, b = 2, so p = log■ 3 ≈ 1.585. Since n is polynomially smaller than n^1.585, Case 1 applies. Thus T(n) = Θ(n^(log■ 3)) ≈ Θ(n^1.585).

9.8 Example 6: T(n) = T(n/2) + n

a = 1, b = 2, p = log■ 1 = 0, so n^p = 1. The function f(n) = n is polynomially larger than 1. The regularity condition holds: a f(n/b) = n/2 ≤ c n for c = 1/2. Case 3 gives T(n) = Θ(n).

9.9 Master Theorem summary table

Recurrence	a, b	n^(log_b a)	Result
T(n)=2T(n/2)+1	2, 2	n	Θ(n)
T(n)=2T(n/2)+n	2, 2	n	Θ(n log n)
T(n)=2T(n/2)+n²	2, 2	n	Θ(n²)
T(n)=4T(n/2)+n²	4, 2	n²	Θ(n² log n)
T(n)=3T(n/2)+n	3, 2	n^1.585	Θ(n^1.585)
T(n)=T(n/2)+n	1, 2	1	Θ(n)

9.10 When the Master Theorem does not apply directly

• Unequal subproblem sizes, such as T(n) = T(n/3) + T(2n/3) + n, do not fit the basic aT(n/b) form.

• Decrement recurrences such as T(n) = T(n–1) + n do not fit the form.

• Some functions f(n) fall between the simple cases or fail the required regularity condition.

• For these, use substitution, recursion trees, or a more general theorem.

> Avoid choosing a case by comparing only exponents without checking the form and conditions. First identify a, b, f(n), and log_b(a).

10. Python Recursion Examples with Complete Outputs

10.1 Factorial

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

Output

```
120
```

The calls are factorial(5) → 5×factorial(4) → 5×4×factorial(3) → 5×4×3×factorial(2) → 5×4×3×2×factorial(1) → 120. There are Θ(n) calls and Θ(n) stack space.

10.2 Fibonacci: naive recursion

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(6))
```

Output

```
8
```

Naive recursion repeats many subproblems. Its runtime is exponential, commonly bounded by O(2^n), and its recursion stack is O(n). Dynamic programming or memoization reduces the time to Θ(n).

10.3 Fibonacci with memoization

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

print(fib_memo(6))
```

Output

```
8
```

Memoization stores solved values so each Fibonacci index is computed once. Time is Θ(n), and the dictionary plus recursion stack use Θ(n) space.

10.4 Binary search

```
def binary_search(a, target):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

values = [2, 5, 9, 13, 18, 21, 30, 41]
print(binary_search(values, 18))
print(binary_search(values, 7))
```

Output

```
4
-1
```

Each iteration discards about half of the remaining search range. Runtime is O(log n) and auxiliary space is O(1) for this iterative implementation.

10.5 Merge sort

```
def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

Output

```
[3, 9, 10, 27, 38, 43, 82]
```

Merge sort has recurrence $T(n) = 2T(n/2) + \Theta(n)$, so time is $\Theta(n \log n)$. This Python version creates slices and result lists, so its extra memory is $\Theta(n)$, aside from recursion-stack overhead.

10.6 Counting work by tracing

When analyzing code, write the values of loop variables and the number of times a statement executes. Do not rely on how fast the program appears on your computer.

```
count = 0
for i in range(1, 5):
    for j in range(i):
        count += 1
print(count)
```

Output

```
10
```

The inner loop runs 1 + 2 + 3 + 4 = 10 times. The general total is n(n+1)/2 for the analogous outer range 1 through n, which is $\Theta(n^2)$.

11. Choosing a Method for a Recurrence

Recurrence pattern	Recommended method	Reason
T(n) = aT(n/b) + f(n), regular conditions	Master Theorem	Fast classification by comparing f(n) with n^(log_b a)
T(n) = T(n–1) + f(n)	Expansion / substitution / sum	Input shrinks by an additive amount
Unequal divisions	Recursion tree or substitution	Basic Master form does not match
Need a formal proof of a guessed bound	Substitution	Induction proves the bound
Need intuition for level contributions	Recursion tree	Shows work by depth

Quick worked comparison

For T(n) = 2T(n/2) + n, all three methods can reach Θ(n log n): substitution proves a guessed bound, a recursion tree shows Θ(n) work at each of Θ(log n) levels, and Master Theorem Case 2 gives the answer directly.

12. Exam-Ready Formula Sheet

Concept	Key result
One loop over n items	Θ(n)
Two complete nested n-sized loops	Θ(n²)
Repeatedly halve n	Θ(log n)
Consecutive work	Add costs; keep dominant term asymptotically
Nested loops	Multiply iteration counts when ranges fully repeat
T(n) = T(n–1) + Θ(1)	Θ(n)
T(n) = T(n–1) + Θ(n)	Θ(n²)
T(n) = T(n/2) + Θ(1)	Θ(log n)
T(n) = 2T(n/2) + Θ(n)	Θ(n log n)
T(n) = aT(n/b) + f(n)	Compare f(n) with n^(log_b a)
Big O	Asymptotic upper bound
Big Ω	Asymptotic lower bound
Big Θ	Tight asymptotic bound
Little o / little ω	Strictly smaller / strictly larger asymptotic growth

13. Practice Questions

Try these before checking a solution. For runtime questions, state your reasoning, not only the final notation.

1. Explain the difference between a data structure and an algorithm. Give two examples of each.

2. Compare a list, set, and dictionary in Python. Which would you choose for membership tests, ordered duplicates, and key-value lookup?

3. Analyze a loop that visits each of n elements once. Explain why its runtime is Θ(n).

4. Determine the complexity of two nested loops, each iterating from 0 to n–1.

5. Find the total number of increments in: for i in range(n): for j in range(i): count += 1.

6. State the formal meaning of O, Ω, and Θ. How does little o differ from Big O?

7. Classify $5n^3 + 2n^2 + 100$ as O, Ω, and Θ using the tightest standard polynomial bound.

8. Solve T(n) = T(n/2) + 1 using expansion or substitution.

9. Solve T(n) = 2T(n/2) + n using a recursion tree.

10. Use the Master Theorem to solve T(n) = 4T(n/2) + n.

11. Use the Master Theorem to solve T(n) = 2T(n/2) + n^2 and verify the regularity condition.

12. Explain why the basic Master Theorem does not directly solve T(n) = T(n–1) + n.

13. Write a Python linear search function and analyze best- and worst-case time.

14. Compare naive recursive Fibonacci with memoized Fibonacci in time and space.

Short answer key

Q	Expected direction
3	Θ(n): constant work repeated n times
4	Θ(n^2): n × n body executions
5	n(n–1)/2 = Θ(n^2)
7	Θ(n^3), hence both O(n^3) and Ω(n^3)
8	Θ(log n)
9	Θ(n log n)
10	a=4, b=2, n^(log■4)=n^2; f(n)=n is smaller, so Θ(n^2)
11	a=2, b=2, comparison term n; f(n)=n^2 dominates and regularity holds, so Θ(n^2)
12	The subproblem size decreases by 1, not by a fixed factor b

14. Final Revision Checklist

• I can explain what data structures are and why the choice of structure affects operation costs.

• I can name common algorithm design paradigms and explain when they are useful.

• I can analyze simple loops, nested loops, and logarithmic loops.

• I can distinguish time complexity, space complexity, and actual wall-clock runtime.

• I can explain best-case, average-case, and worst-case analysis.

• I can define Big O, Big Omega, Big Theta, little o, and little omega.

• I can expand a recurrence and use substitution to prove a bound.

• I can total work per level in a recursion tree.

• I can identify a, b, and f(n) and apply the three standard Master Theorem cases.

• I can write and trace the included Python examples and explain their outputs.

Study advice: For every recurrence problem, first write down the recurrence clearly, identify its base case, and state any assumptions (for example, n is a power of two). Then choose the method. Always show intermediate steps in exam answers.